

Question

Design and Implementation of a Blockchain-Based Library Book Lending Tracker

Assignment Objectives

At the end of this lab assignment, students should be able to:

1. Understand the basic structure and operation of a blockchain.
2. Implement a simple blockchain in C for application data storage.
3. Integrate cryptographic hash functions (SHA-256) to ensure data integrity.
4. Implement user authentication and access control mechanisms.
5. Ensure data persistence using file-based storage.
6. Apply proper error handling and data validation techniques.

Assignment Description

Traditional library lending records can be manipulated, a librarian can quietly mark a lost book as "returned," or a borrower can dispute what the paper log shows. Blockchain eliminates this by making every borrow and return event immutable and cryptographically verifiable.

Assignment Instructions: Requirements

Your Library Lending system must implement the following:

1. A blockchain data structure (blocks containing lending records)
2. A book registry and a member registry, each loaded from file, to validate book IDs and member IDs before any lending action is recorded
3. Cryptographic hashing (SHA-256) to link blocks
4. Digital signatures to authenticate lending actions
5. Chain validation logic to detect tampering
6. A simple CLI interface to borrow, return, and view lending records

Book and Member Registries

Before any lending action can be recorded, the program must load two registries from file: books.txt and members.txt:

books.txt — one book per line, in the following format:

1. BK001,Things Fall Apart,Chinua Achebe
2. BK002,Americanah,Chimamanda Ngozi Adichie
3. BK003,The River Between,Ngũgĩ wa Thiong'o

Fields: book_id, title, author

members.txt — one member per line, in the following format:

- ALU001,John Doe,BLK101

```
- ALU002,Jane Smith,BLK101
- ALU003,Amara Diallo,BLK101
```

Fields: member_id, full_name, course_code

The program must:

- Load all books and all members into memory at startup (arrays of structs)
- Reject any borrow or return attempt referencing a book_id or member_id not found in the registries
- Print an error message if either books.txt or members.txt is missing or empty

Book Struct:

```
typedef struct {
    char book_id[20];
    char title[80];
    char author[50];
} Book;
```

Member Struct:

```
typedef struct {
    char member_id[20];
    char full_name[50];
    char course_code[10];
} Member;
```

Block Structure

Each block in your attendance chain must contain the following fields:

Field	Type	Description
index	int	The block's position in the chain (0 = Genesis Block)
timestamp	time_t	Unix timestamp when the block was created
book_id	char[20]	Must match a valid ID in the book registry

book_title	char[80]	Copied from the book registry at the time of the transaction
member_id	char[20]	Must match a valid ID in the member registry
Member_name	char[50]	Copied from the member registry at the time of the transaction
action	char(10)	BORROWED, RETURNED, or OVERDUE
previous_hash	char[65]	SHA-256 hash of the previous block
signature	unsigned char[72]	ECDSA digital signature of the block data
hash	char[65]	SHA-256 hash of all the above fields combined

Minimum Functional Requirements

- **Load Registries** — At startup, read books.txt and members.txt into arrays of Book and Member structs. Validate that both files exist and are not empty.
- **Genesis Block** — Your chain must start with a genesis block (index 0) with previous_hash set to 64 zeros.
- **Borrow Book** — Before creating a block, look up both the book ID and the member ID in their respective registries. If either is not found, print ERROR: Book or Member not found and abort. If both are found, and the book is not already on loan, create the block, sign it, compute its hash, and append it to the chain.
- **Return Book** — Look up the book's most recent BORROWED entry on the chain. If none exists (the book was never borrowed, or was already returned), print an error. Otherwise create a new RETURNED block, sign it, compute its hash, and append it to the chain.
- **Validate Chain** — Iterate through all blocks and verify (a) each block's hash is correct and (b) each block's previous_hash matches the preceding block's hash.
- **View Records** — Print all lending records to the terminal including book title, member name, action, timestamp, and signature validity.

- **Tamper Detection** — Demonstrate that modifying a past block breaks chain validation..

Deliverables:

- Complete C source code for each task presenting the GitHub link.
- Demo Video (3–5 mins) — Screen recording showing: loading the registry, marking valid and invalid student IDs, viewing records, validating the chain, and demonstrating tamper detection.
- **Technical Report (PDF/Word)**
 - Introduction
 - Description of blockchain implementation
 - Security mechanisms (SHA-256, authentication)
 - Data persistence approach
 - Error handling strategy
 - Screenshots of application execution
 - Challenges encountered and solutions
 - **System Design Diagram** — A diagram showing your block structure, chain linkage, and how the registry connects to the attendance flow.

● README File

- Compilation instructions
- How to build and run the application
- Required libraries and dependencies

Rubric

Assignment 1 Rubric

Criteria	Ratings	Pts
This criterion is linked to a Learning	4 to >3.5 pts Exemplary Meets all stated requirements	4 pts
	3.5 to >2.5 pts Proficient Meets 85% to 90% of the stated requirements	
	2.5 to >1.59 pts Developing Meets 60% to 85% of the stated requirements	
	1.59 to >0.0 pts Beginning Meets 50% to 60% of the stated requirements	
	0 pts No Marks Meets less than 50% of the stated requirements	

Outcome Compliance with Submission Requirements		
This criterion is linked to a Learning Outcome Technical Correctness and Code Quality	4 to >3.5 ptsExemplaryCode is correct, well-defined structures, high quality 3.5 to >2.5 ptsProficientCode is mostly correct, well-defined structures, good quality 2.5 to >1.6 ptsDevelopingCode is somewhat correct, partially defined structures, fair quality 1.6 to >0.0 ptsBeginningCode has significant issues, poorly defined structures, poor quality 0 ptsNo MarksCode is incorrect, undefined structures, very poor quality	4 pts
This criterion is linked to a Learning Outcome Security and	4 to >3.5 ptsExemplaryRobust security, demonstrate proper use of cryptographic keys 3.5 to >2.5 ptsProficientGood security, mostly proper use of cryptographic keys 2.5 to >1.59 ptsDevelopingSatisfactory security, some proper use of cryptographic keys 1.59 to >0.0 ptsBeginningFair security, minimal use of cryptographic keys 0 ptsNo MarksPoor security, improper or no use of cryptographic keys	4 pts

Use of Cryptographic Keys		
This criterion is linked to a Learning Outcome Demo Video	3 to >2.62 pts Exemplary Video clearly demonstrates all required functionalities: registry loading, attendance marking, invalid ID handling, chain validation, and tamper detection. Explanation is professional and easy to follow. 2.62 to >1.87 pts Proficient Video demonstrates most required functionalities with minor clarity or presentation issues. 1.87 to >1.19 pts Developing Video incomplete, unclear, or missing important demonstrations. 1.19 to >0.0 pts Beginning Video missing or fails to demonstrate the system properly. 0 pts No Marks Missing video	3 pts

Total Points: 15